



31 DAY CHALLENGE



31 Days of React Design Patterns

DAY

1

Day 1 — Introduction to React Design Patterns



Learning React Design Patterns one day at a time.



TODAY'S GOAL:

Understand what design patterns are and why React developers use them.



```
// App.jsx
function App() {
  return (
    <Layout>
      <Header />
      <Main />
      <Footer />
    </Layout>
  );
}
```



COMPONENT TREE



PROGRESS

1/31



YOUR 31-DAY JOURNEY



1

Day 1

2

Day 2

3

Day 3

4

Day 4

5

Day 5

...

31

Day 31



PRACTICAL
EXAMPLES



REAL-WORLD
PATTERNS



BETTER CODE
ARCHITECTURE



LEVEL UP YOUR
REACT SKILLS

Why Do We Need Design Patterns?

```
function App() {  
  return (  
    <Layout>  
      <Header />  
      <Main />  
    </Layout>  
  );  
}
```

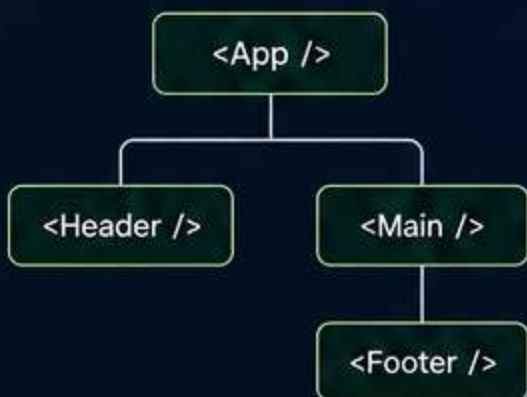
```
function App() {  
  return (  
    <Provider>  
      <Router>  
        <Layout> >  
          <Header />  
          <Sidebar />  
          <Content />  
          <Footer />  
        </Router>  
      </Provider>  
    );  
}
```



When Projects Start Simple

- ✓ Few components
- ✓ Minimal state
- ✓ Easy to understand

SIMPLE COMPONENT TREE



Clean, easy to manage and scale.



Small App



More Features



More Components



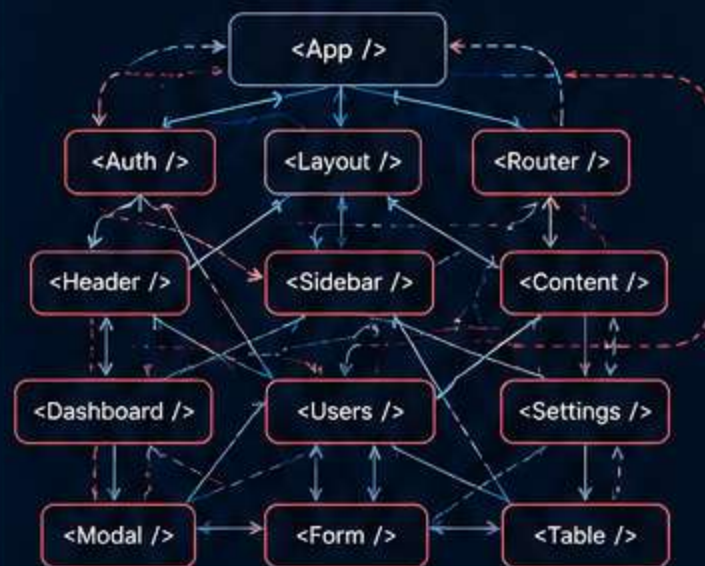
Growing Complexity



As Applications Grow

- ✗ Components become huge
- ✗ Logic gets duplicated
- ✗ State becomes messy
- ✗ Maintenance becomes harder

COMPLEX COMPONENT TREE



Hard to manage, easy to break, difficult to scale.



This is where Design Patterns help.

They give us proven solutions to build scalable, maintainable, and clean React applications.

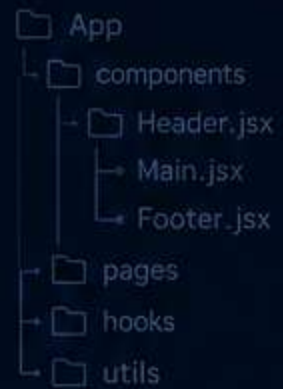


DAY
1

31 Days of
React Design Patterns

```
function App() {
  return (
    <div>
      <Header />
      <Main />
      <Footer />
    </div>
  );
}
```

What Are React Design Patterns?

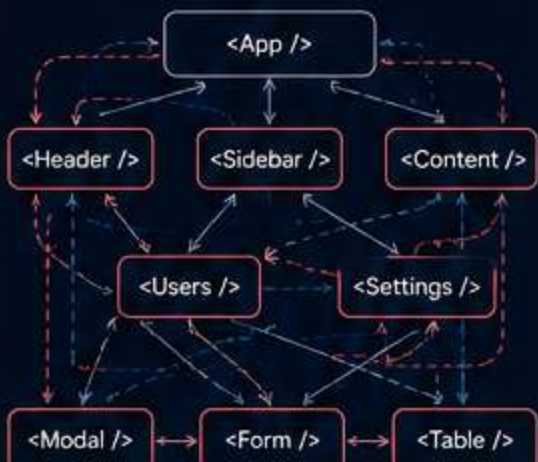


React Design Patterns are
reusable solutions to recurring
problems in React applications.



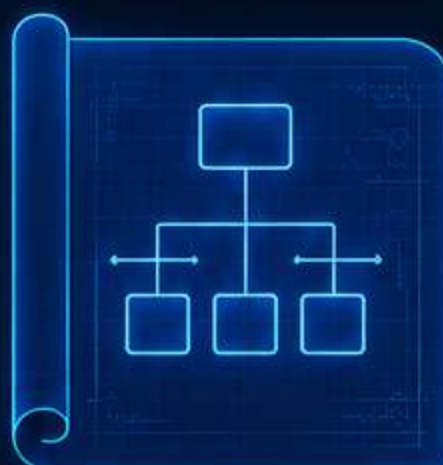
Problem

Code becomes tangled,
hard to maintain.



Pattern

Apply a proven approach
to structure your code.



Clean Solution

Well-structured, scalable,
and easy to maintain.



Not Libraries



They are not packages you install.
They are ideas and structures.



Not Frameworks



They don't dictate how you build.
You stay in control.



**Approaches for
Organizing Code**



They help you write cleaner, scalable,
and maintainable React applications.



Key Takeaway

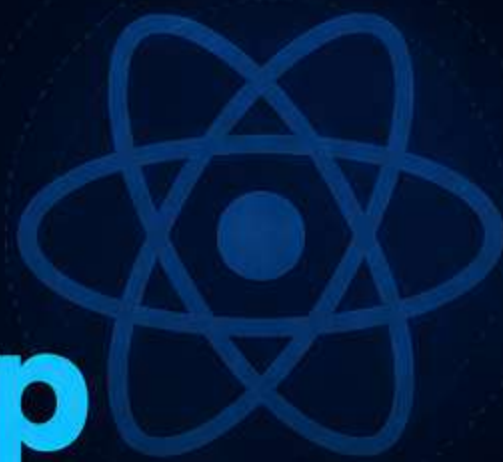
Design patterns bring **clarity** to complexity. They help your codebase **grow** without falling apart.



DAY
1

31 Days of
React Design Patterns

How Design Patterns Help



```
function App() {
  return (
    <div>
      <Header />
      <Main />
      <Footer />
    </div>
  );
}
```

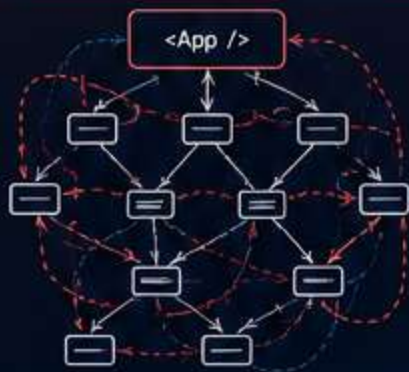


Think of Design Patterns as
architectural blueprints for
your React code.

```
const [state, setState]
  = useState();
useEffect(() => {
  fetchData();
}, []);
```



Problem

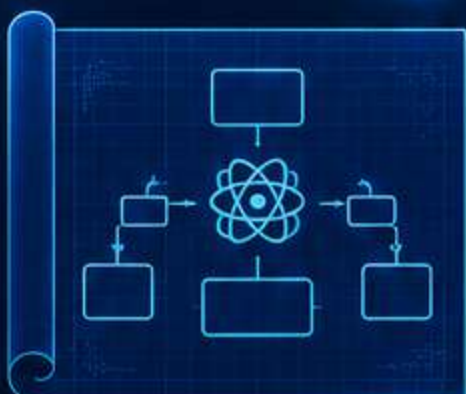


- ⚠ Messy component tree
- ⚠ Scattered state
- ⚠ Duplicate code
- ⚠ Hard to maintain

```
useState(...)
useEffect(...)
fetchData()
```



Pattern



- 📦 Reusable structure
- 🛡 Proven approach
- 🔄 Solves recurring problems
- 🏗 Architecture for your code



Structured
Solution



- ✓ Organized components
- ✓ Centralized state
- ✓ Reusable logic
- ✓ Clean and maintainable
- ✓ Easy to scale

```
components/
hooks/
context/
utils/
pages/
```

BENEFITS



Separate Concerns

Keep UI, logic, and state management well separated.



Reuse Logic

Build once, use anywhere. Save time and avoid repetition.



Improve Readability

Well-structured code is easier to read, understand, and collaborate on.



Reduce Duplication

Eliminate repeated code and keep your codebase DRY.



Scale Applications Confidently

Handle growth and complexity with a solid architecture.



Key Takeaway

Design patterns turn **chaos** into **clarity**. They help you build React applications that are clean, maintainable, and scalable.



Without Structure vs With Structure

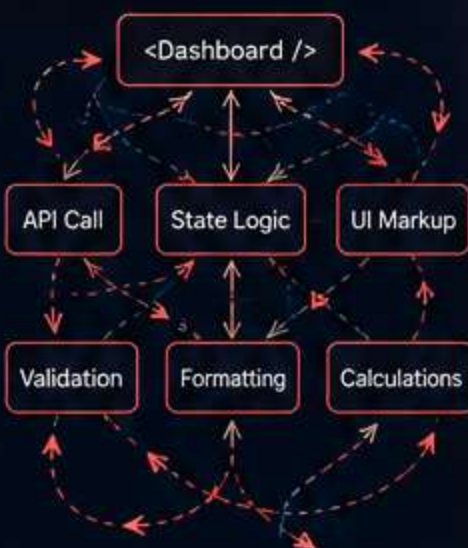
```
function App() {  
  return (  
    <div>  
      <Header />  
      <Main />  
      <Footer />  
    </div>  
  );  
}
```



Everything Inside One Component

```
Dashboard.jsx ×  
1 function Dashboard() {  
2   // API calls  
3   // State  
4   // UI  
5   // Business logic  
6   return (  
7     <div> ... </div>  
8   )  
}
```

- ⚠ Mixed concerns
- ⚠ Scattered state
- ⚠ Hard to test
- ⚠ Hard to maintain
- ⚠ Difficult to scale



Too many responsibilities

Chaos

Clarity



Responsibilities Separated

```
Dashboard.jsx ×  
1 function Dashboard() {  
2   return <UserProfile />;  
3 }
```

Component Hierarchy



Folder Structure

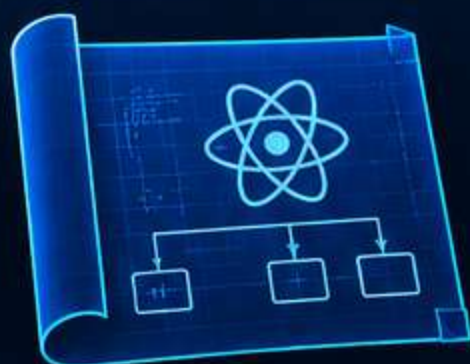
```
components/  
├── UserProfile.jsx  
├── StatsCard.jsx  
└── ActivityList.jsx  
  
hooks/  
services/  
utils/
```

Reusable Components

<UserProfile />	✓
<StatsCard />	✓
<ActivityList />	✓
...	

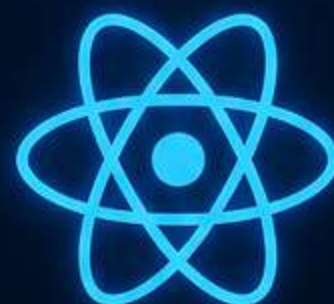


Single responsibility,
better organization



Patterns help move from **chaos** to **clarity**.

Clean structure. Maintainable code.
Scalable applications.



```
const [state, setState] = useState();  
useEffect(() => {  
  fetchData();  
});
```


DAY
1

31 Days of
React Design Patterns



Benefits of Design Patterns

```
function App() {  
  return (  
    <div>  
      <Header />  
      <Main />  
      <Footer />  
    </div>  
  );  
}
```

```
src/  
├── components/  
│   ├── Header.jsx  
│   ├── Main.jsx  
│   └── Footer.jsx  
├── hooks/  
├── utils/  
├── pages/  
└── App.jsx
```



01

Easier Maintenance

Applications become easier to update and debug.



03

Cleaner Code

Improve readability and create well-organized components.



05

Faster Onboarding

New developers can understand the codebase more quickly.



02

Better Scalability

Grow your application without increasing complexity.



04

Improved Teamwork

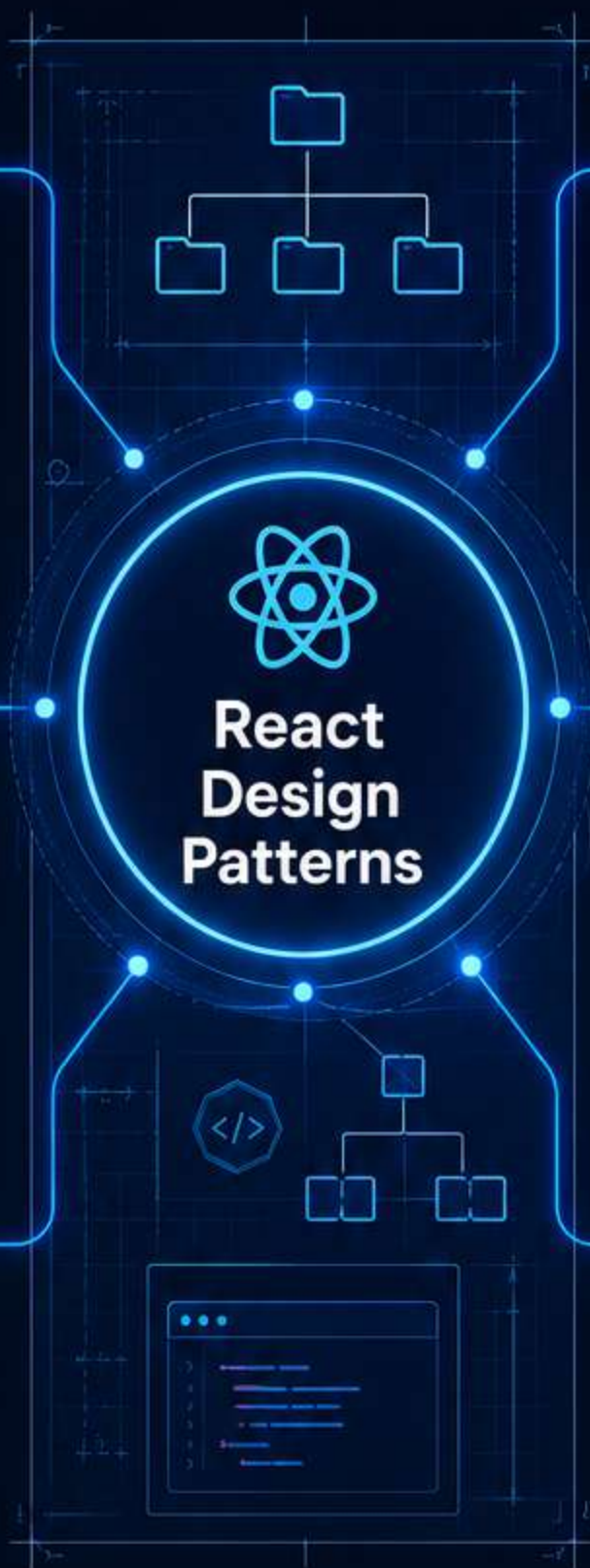
Consistent patterns make collaboration easier.



06

More Reusable Components

Reduce duplication and maximize code reuse.



Build applications that are
easier to maintain, scale, and understand.



DAY
1

31 Days of
React Design Patterns

When Should You Learn Patterns?

```
function App() {
  return (
    <div>
      <Header />
      <Main />
      <Footer />
    </div>
  );
}
```

BEGINNER



Building Medium-to-Large Apps

As projects grow, patterns help maintain structure and scalability.



Working with Teams

Shared architecture improves collaboration and consistency.

INTERMEDIATE



Reusing Logic Frequently

Patterns help eliminate duplication and encourage reusability.



Managing Growing Complexity

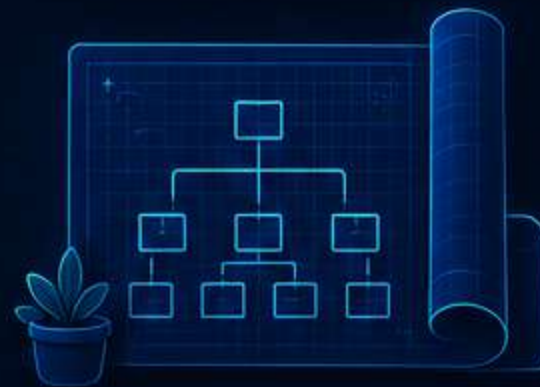
Structure becomes increasingly important as features expand.

ADVANCED



Preparing for Professional React Roles

Strong architectural skills are essential for real-world applications.



Beginner



Growing Projects



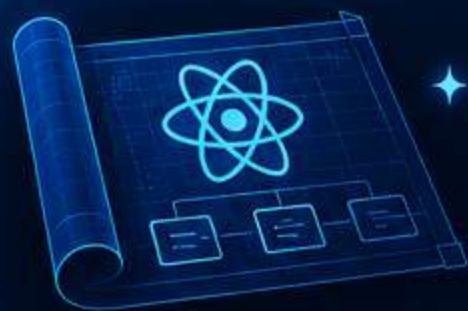
More Complexity



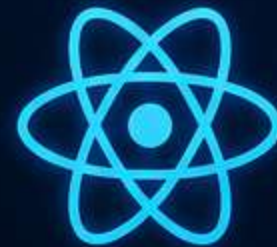
Design Patterns



Professional Developer



Learn patterns when **complexity grows**—not before.



DAY
1

31 Days of
React Design Patterns



Don't Over Engineer

Small App



Just what you need.

Simple.
Clean.
Effective.



Good Code



Fancy Code

Over Engineered



Too much for too little.



src/
components/
Header.jsx
Stats.jsx
Chart.jsx
pages/
Dashboard.jsx
App.jsx
index.jsx

- ✓ Few Components
- ✓ Simple Structure
- ✓ Minimal State
- ✓ Easy to Maintain

src/
core/
di/
factory/
provider/
service/
features/
todo/
components/
hooks/
store/
api/
utils/
shared/
components/
hooks/
utils/
constants/
types/
infrastructure/
api/
config/
adapters/
middleware/
patterns/
observer/
strategy/
command/
decorator/
state/
global/
atoms/
selectors/
slices/

- ⚠ Huge Component Tree
- ⚠ Over Abstraction
- ⚠ Complex Structure
- ⚠ Hard to Maintain
- ⚠ Slower Development

Why is this
so complicated
for a TODO
app?!



Building Tiny
Projects

Patterns may introduce
unnecessary complexity.



Solving Simple
Problems

Simple solutions are
often the best solutions.



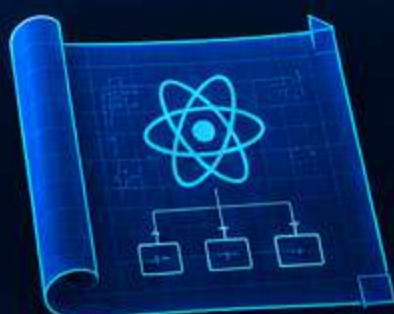
Learning React
Fundamentals

Master the basics
before advanced
architecture.



When the Pattern
Adds More
Complexity Than Value

Avoid abstraction for
abstraction's sake.



Patterns are **tools** not **requirements**.

Use them only when they solve a **real problem**.



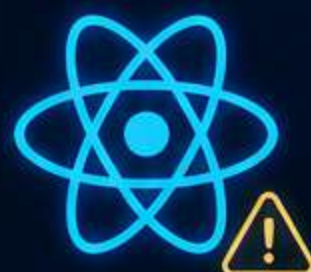
Solve problems



Add value



Avoid complexity
for no reason



```
function App() {  
  return (  
    <div>  
      <SmallApp />  
    </div>  
  );  
}
```



```
const OverEngineeredApp = () => {  
  return (  
    <ComplexArchitectureProvider>  
      <EverythingProvider>  
        <App />  
      </EverythingProvider>  
    </ComplexArchitectureProvider>  
  );  
}
```


Where You'll See Them

```
function App() {  
  return (  
    <Dashboard />  
  )  
}
```



E-Commerce Apps

Product listings, shopping carts, checkout flows, and filters.



Dashboards

Analytics panels, charts, widgets, and data tables.



SaaS Platforms

Admin panels, user management, subscriptions, and settings pages.



React Design Patterns

THE PATTERN ROADMAP



Social Media Apps

News feeds, profiles, notifications, comments, and messaging interfaces.



Enterprise Systems

Complex workflows, reports, permissions, and large scale applications.



01



Container Pattern

Manage logic and state outside presentational components.

02



Compound Components

Build flexible and reusable component compositions.

03



Render Props

Share code between components using functions.

04



Custom Hooks

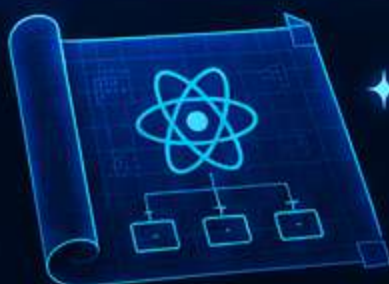
Extract logic into reusable functions with Hooks.

05



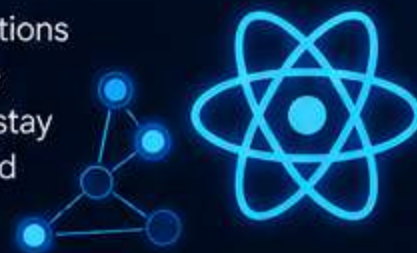
Provider Pattern

Share state and logic across the component tree.

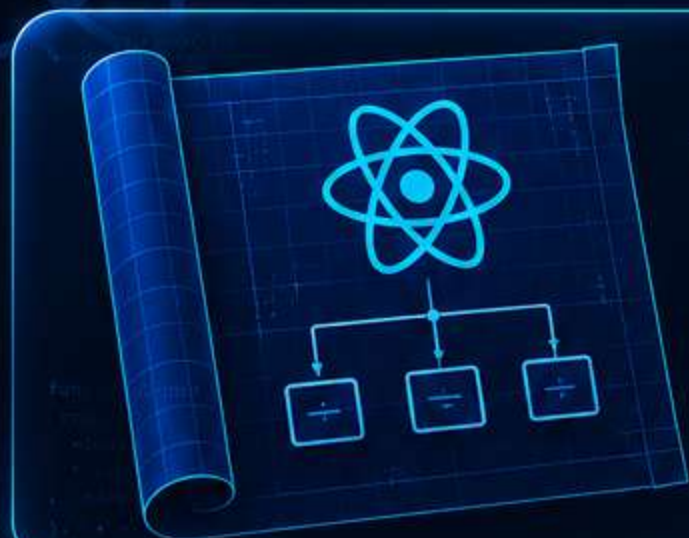


Patterns are everywhere you just don't notice them.

Modern applications rely on reusable architecture to stay maintainable and scalable.



Today's Lesson



Design Patterns are not shortcuts.

They are proven ways to organize React applications.



THE JOURNEY



Day 1

Introduction to React Design Patterns



Learn Patterns

Explore and understand proven approaches.



Understand Trade offs

Choose the right pattern for the right problem.



Build Better Apps

Apply patterns to build clean and scalable apps.



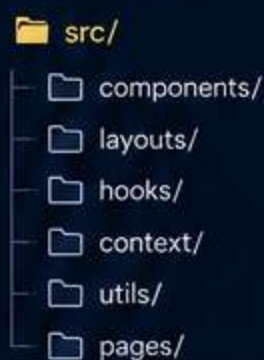
Become a Better React Developer

Level up your skills and architect with confidence.



My Goal Over the Next 31 Days

Learn when to use **React Design Patterns** and when not to.



Growth Through Better Architecture



Question

What's the biggest challenge you've faced while structuring React projects?



Follow for More



@mtayyabrawan

Follow my 31 Days of React Design Patterns journey.



Learn Consistently



Apply Practically



Build Confidently



Grow Continuously